

Useful Python Libraries

Kayla Wilhoit

BIO 724

April 7 2026

Biopython



<https://biopython.org/docs/latest/index.html>

- Biopython is a set of libraries and modules designed to streamline reading and using biological data in Python
 - Deals with file parsing, etc. so you can go straight to the fun analysis
 - Also has several built in modules for running BLAST and accessing databases
- One of the best tools is the **Bio.SeqIO** module
 - Allows very easy reading and writing of sequence data (fasta, genbank, etc.)
 - Can combo with gzip module for gzipped data
 - Can read large sequences into dictionaries for efficient access

```
from Bio import SeqIO

for seq_record in SeqIO.parse("ls_orchid.fasta", "fasta"):
    print(seq_record.id)
    print(repr(seq_record.seq))
    print(len(seq_record))
```

```
>>> from Bio import SeqIO
>>> identifiers = [seq_record.id for seq_record in SeqIO.parse("ls_orchid.gbk", "genbank")]
>>> identifiers
['Z78533.1', 'Z78532.1', 'Z78531.1', 'Z78530.1', 'Z78529.1', 'Z78527.1', ..., 'Z78439.1']
```

```
>>> from Bio import SeqIO
>>> orchid_dict = SeqIO.to_dict(SeqIO.parse("ls_orchid.gbk", "genbank"))
```

```
>>> import gzip
>>> from Bio import SeqIO
>>> with gzip.open("ls_orchid.gbk.gz", "rt") as handle:
...     print(sum(len(r) for r in SeqIO.parse(handle, "gb")))
...
67518
```

The **Seq** object

- The **Seq** object is a fancy string with extra built-in methods specific for sequences
 - **gc_fraction**
 - **translate** (using NCBI codon tables)
- These methods are tested, useful, and customizable – don't reinvent the wheel!
- The **Seq** object is also immutable
 - Helps avoid editing your raw sequence data!
 - You can make a **MutableSeq** if needed

```
>>> from Bio.Seq import Seq
>>> from Bio.SeqUtils import gc_fraction
>>> my_seq = Seq("GATCGATGGGCCTATATAGGATCGAAAATCGC")
>>> gc_fraction(my_seq)
0.46875
```

```
>>> from Bio.Seq import Seq
>>> messenger_rna = Seq("AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCCGAUAG")
>>> messenger_rna
Seq('AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCCGAUAG')
>>> messenger_rna.translate()
Seq('MAIVMGR*KGAR*')
```

```
>>> coding_dna.translate(table=2, stop_symbol="@")
Seq('MAIVMGRWKGAR@')
```

The **SeqRecord** class

- A **SeqRecord** is a container that holds a **Seq** object and a bunch of info related to it
 - Very useful for detailed annotations such as GenBank files
 - Includes fields like reference where you can add the citation of the data!
- Can just be the **Seq** object – when using **SeqIO** on plain fasta files
 - The **SeqRecord.format()** method can output the string in a specific format (fasta)
- Uses **SeqFeature** to describe positions and ranges of data
 - Can be exact or “fuzzy” – useful for open-ended analysis!
 - Useful for setting a new region of interest which you can use or query later

```
>>> from Bio import SeqFeature
>>> start_pos = SeqFeature.AfterPosition(5)
>>> end_pos = SeqFeature.BetweenPosition(9, left=8, right=9)
>>> my_location = SeqFeature.SimpleLocation(start_pos, end_pos)
```

```
>>> from Bio import SeqIO
>>> my_snp = 4350
>>> record = SeqIO.read("NC_005816.gb", "genbank")
>>> for feature in record.features:
...     if my_snp in feature:
...         print("%s %s" % (feature.type, feature.qualifiers.get("db_xref")))
```

The **Alignment** object

- Contains sequences aligned to each other – which can be in **Seq** or **SeqRecord** format!
 - Sequences in a pairwise alignment will have query and target attributes
 - Alignments are iterable objects!
- Also has specific methods associated such as counts and flags (especially useful for SAM/BAM flags)

```
>>> from Bio import Align
>>> alignments = Align.parse("ex1.sam", "sam")
```

```
>>> if alignment.flag & 4:
...     print("unmapped")
... else:
...     print("mapped")
```

```
>>> for alignment in alignments:
...     print(alignment.target.id)
```

```
>>> counts = pairwise_alignment.counts()
>>> counts
<AlignmentCounts object (5 aligned letters; 4 identities; 1 mismatches; 3 gaps) at ...>
```

Bonus – list comprehensions and BLAST

- The Biopython built-in methods and readable functions make list comprehensions an easy and fast way to process sequences without hours of setup

```
>>> from Bio import SeqIO
>>> records = (
...     rec.reverse_complement(id="rc_" + rec.id, description="reverse complement")
...     for rec in SeqIO.parse("ls_orchid.fasta", "fasta")
...     if len(rec) < 700
... )
>>> SeqIO.write(records, "rev_comp.fasta", "fasta")
18
```

- Run BLAST on the internet or locally using a **Seq** or **SeqRecord** object!
 - Accesses NCBI blast url by default (requires internet connection)
 - Can run locally against your own database (faster, no internet needed)
 - Stores the results as a **BlastRecord** object (a type of Python list) which also has its own set of methods and can be sliced, indexed, etc.
 - The individual **Hits** within the list are also Biopython **Alignment** objects!

```
>>> from Bio import Blast
>>> from Bio import SeqIO
>>> record = SeqIO.read("m_cold.fasta", "fasta")
>>> result_stream = Blast.qblast("blastn", "nt", record.seq)
```

```
>>> blast_record[0] # retrieves the top hit
<Bio.Blast.Hit target.id='gi|262205317|ref|NR_030195.1|' query.id='42291'; 1 HSP>
>>> blast_record[-1] # retrieves the last hit
<Bio.Blast.Hit target.id='gi|397513516|ref|XM_003827011.1|' query.id='42291'; 1 HSP>
```

Cool biopython scripts with explanations

- Extracting intergenic regions from a genbank file and fasta
 - <http://bytesizebio.net/2010/02/11/short-bioinformatic-hacks-reading-between-the-genes/>
- Reading a compressed fastq file from Unix pipes
 - https://biopython.org/wiki/Reading_from_unix_pipes
- Find query sequences with no BLAST match
 - https://biopython.org/wiki/Retrieve_nonmatching_blast_queries
- Clean up a sequence (filter length, duplicates)
 - https://biopython.org/wiki/Sequence_Cleaner
- Retrieve Entrez Gene IDs
 - https://biopython.org/wiki/Annotate_Entrez_Gene_IDs
- Concatenating single gene NEXUS alignments into a supermatrix
 - https://biopython.org/wiki/Concatenate_nexus

Pysam and Pybedtools

- Wrappers and extensions for the command-line **samtools** and **bedtools**
- Includes many flag-based functions as methods of an alignment object
- My favorites:
 - Pysam **get_aligned_pairs** for getting positions in both query and reference
 - Pybedtools **intersect** for finding if my SNPs fall within specific regions
- Used in conjunction with the **multiprocessing** library to process each chromosome in parallel within an alignment
 - Use caution! Can be memory intensive

```
from pysam import VariantFile

bcf_in = VariantFile("test.bcf") # auto-detect input format
bcf_out = VariantFile('-', 'w', header=bcf_in.header)

for rec in bcf_in.fetch('chr1', 100000, 200000):
    bcf_out.write(rec)
```

```
from pybedtools import BedTool

snps = BedTool('snps.bed.gz') # [1]
genes = BedTool('hg19.gff') # [1]

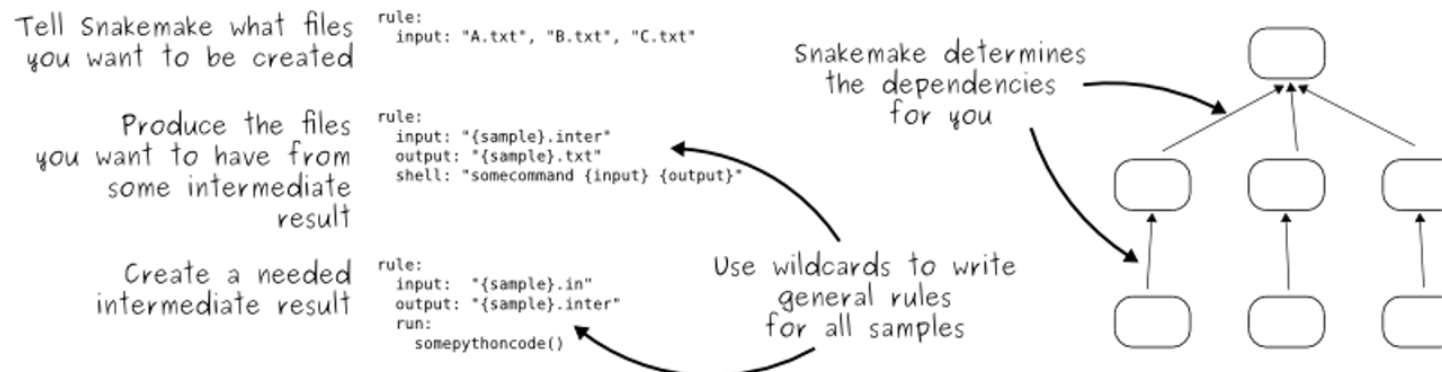
intergenic_snps = snps.subtract(genes) # [2]
nearby = genes.closest(intergenic_snps, d=True, stream=True) # [2, 3]

for gene in nearby: # [4]
    if int(gene[-1]) < 5000: # [4]
        print gene.name # [4]
```


Snakemake



- Snakemake is a workflow management system that is python-based
 - Similar to **nextflow**
- Snakemake works as a series of **rules** that can wrap commands or scripts, and define specified input and output files
- The strength of snakemake – based on the defined inputs and outputs, it will **only run rules when needed** and **run the next rule as soon as the input data is available**



```

configfile: "config.yaml"

rule all:
  input:
    expand(
      "results/plots/{country}.hist.pdf",
      country=config["countries"]
    )

rule download_data:
  output:
    "data/worldcitiespop.csv"
  log:
    "logs/download.log"
  conda:
    "envs/curl.yaml"
  shell:
    "curl -L https://burntsushi.net/stuff/worldcitiespop.csv > {output} 2> {log}"

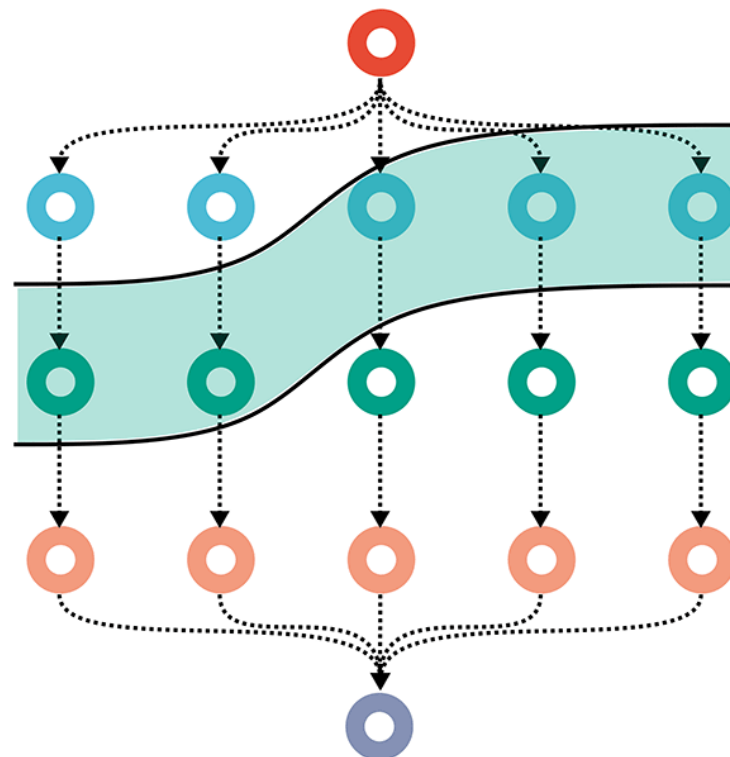
rule select_by_country:
  input:
    "data/worldcitiespop.csv"
  output:
    "results/by-country/{country}.csv"
  log:
    "logs/select-by-country/{country}.log"
  conda:
    "envs/xsv.yaml"
  shell:
    "xsv search -s Country '{wildcards.country}' "
    "{input} > {output} 2> {log}"

rule plot_histogram:
  input:
    "results/by-country/{country}.csv"
  output:
    "results/plots/{country}.hist.svg"
  container:
    "docker://faizanbashir/python-datascience:3.6"
  log:
    "logs/plot-hist/{country}.log"
  script:
    "scripts/plot-hist.py"

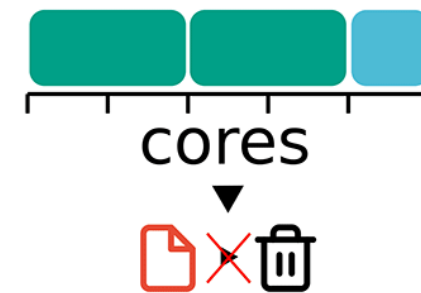
rule convert_to_pdf:
  input:
    "{prefix}.svg"
  output:
    "{prefix}.pdf"
  log:
    "logs/convert-to-pdf/{prefix}.log"
  wrapper:
    "0.47.0/cairosvg"

```

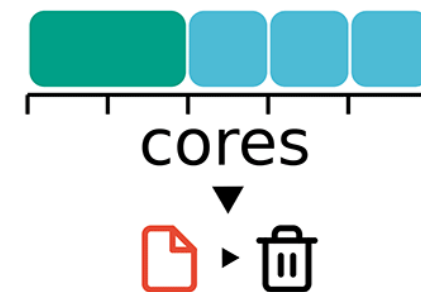
a



b



c



```

rule bwa_map:
  input:
    "data/genome.fa",
    "data/samples/{sample}.fastq"
  output:
    "mapped_reads/{sample}.bam"
  shell:
    "bwa mem {input} | samtools view -Sb - > {output}"

```

```

rule samtools_sort:
  input:
    "mapped_reads/{sample}.bam"
  output:
    "sorted_reads/{sample}.bam"
  shell:
    "samtools sort -T sorted_reads/{wildcards.sample} "
    "-O bam {input} > {output}"

```

Why use Snakemake?

- **Reproducibility** – your workflows will be readable, shareable, and consistent
- **Ease of use** – You can tweak a few parameters/scripts and re-run the workflow at the press of a button – and only the affected rules will be re-run!
- **Parallelization** – Snakemake lets you intuitively parallelize running the same steps on multiple files without having to reformat your scripts to work with parallel or multiprocessing
- **Flexibility** – You can start writing your pipeline as separate pieces for testing and then quickly turn it into a Snakemake workflow with minimal adjustments afterwards

Snakemake info and tutorials

- Official Snakemake tutorial
 - <https://snakemake.readthedocs.io/en/stable/tutorial/basics.html>
- Catalog of quality snakemake workflows
 - <https://snakemake.github.io/snakemake-workflow-catalog/>
- Carpentries tutorial for cDNA transcriptome alignment
 - <https://carpentries-incubator.github.io/snakemake-novice-bioinformatics/index.html>
- Running Snakemake with SLURM (like on the DCC)
 - <https://hackmd.io/@bluegenes/BJPrj7WB>

Extras – click and Xonsh



Click

- **click** is a package for turning Python scripts and libraries into command-line accessible functions
 - You can add flags, a help file, and more
 - Great for sharing scripts among lab members or in a pipeline
- **Xonsh** is a shell that allows you to mix Python and Bash syntax on the command line or in scripts
 - Easily run shell commands within a python loop or on a pandas data frame
 - Can behave oddly at times

```
(genomics) pmagwene@thingtwo:/FastData/pmagwene/tmp$ python lof2.py --help
Usage: lof2.py [OPTIONS] INFILE OUTFILE

Identify genes with candidate LOF alleles due to premature stop codons.

Identify most common amino acid sequence among translated CDS seqs; call
this the modal sequence. Return any AA sequence whose predicted length is
less than threshold * length of the modal sequence.

Options:
  --threshold FLOAT  Fractional length threshold for identifying truncated
                      seqs
  --help             Show this message and exit.
```



```
(base) pmagwene@darkbook ~ % fasta_files = $(ls *.fasta)
(base) pmagwene@darkbook ~ % fasta_files
'Bt63_genome_FungalPop_202410122.fasta\nCneo_KDZ_transposons.aligned.fasta\nKDZ7_core_aligned.fasta\n
LPD1alpha_references.fasta\nNC1_CDC1_prot.fasta\nNC1_CFT1_prot.fasta\nNC1_CNAG_05324_cds.fasta\nNC1_C
NAG_05324_prot.fasta\nNC1_CNAG_06934_cds.fasta\nNC1_CNAG_06934_prot.fasta\nNC1_LMP1_cds.fasta\nNC1_LM
P1_prot.fasta\nNC1_YOX101_cds.fasta\nNC1_YOX101_prot.fasta\nch4explore.aligned.fasta\nch4explore.fasta
\nchr13peak.aligned.fasta\nfull_KDZ_transposons.aligned.fasta\n'
(base) pmagwene@darkbook ~ % fasta_files.split("\n")[:5]
['Bt63_genome_FungalPop_202410122.fasta',
'Cneo_KDZ_transposons.aligned.fasta',
'KDZ7_core_aligned.fasta',
'LPD1alpha_references.fasta',
'NC1_CDC1_prot.fasta']
```